2023

Глава 2 Управляющие конструкции

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Лектор: д.т.н., проф.
Иванова Галина Сергеевна

2.1 Блок операторов

Блок операторов используется в конструкциях ветвления, выбора и циклов, предусматривающих один оператор.

Формат:

$ Оператор = Оператор | {{Оператор}}

Пример:

```
{
 f = a + b;
 a += 10;
}
```

Точка с запятой является частью оператора.

2.2 Оператор условной передачи управления

Оператор условной передачи управления используется при обработке вариантов вычислений и реализует конструкцию ветвления.

$ **Оператор_if** = `if` (Выражение) Оператор [`else` Оператор]

Оператор – простой оператор или блок операторов в операторных скобках {…}

Выражение – булево или целочисленное выражение:

- если результат выражения `true`, то выполняется оператор, следующий за выражением;
- если результат выражения `false`, то либо выполняется оператор альтернативной ветви, либо (при отсутствии альтернативной ветви) управление передается следующему за `if` оператору.

Пример. Разработать программу, которая вводит число от 0 до 15 и выводит его шестнадцатеричный эквивалент, например:

8 ⇒ ‘8’

12 ⇒ ‘C’

Кодирование символов

Таблица 1.1. Базовая таблица кодировки ASCII

32	пробел	48	0	64	@	80	P	96	`	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	|
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	

В соответствии с таблицей:

'0' – 30_{16} === 48_{10} тогда символ **`ch ='0'+n;`**

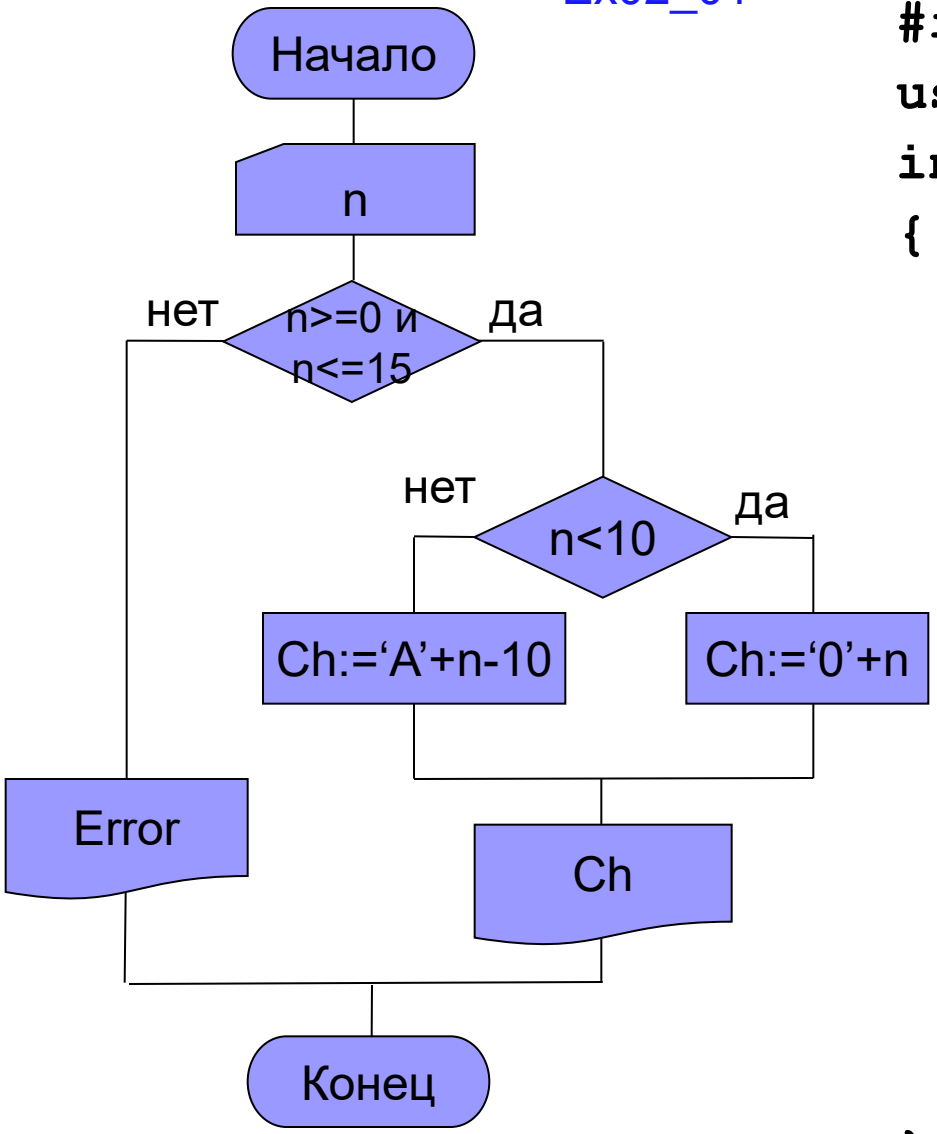
'A' – 41_{16} === 65_{10} тогда символ **`ch ='A'+n-10;`**

Пример:

Пусть введено 13, тогда ch = 65+13-10 = 68 = 'D'

Программа отображения 16-тиричных эквивалентов

Ex02_01

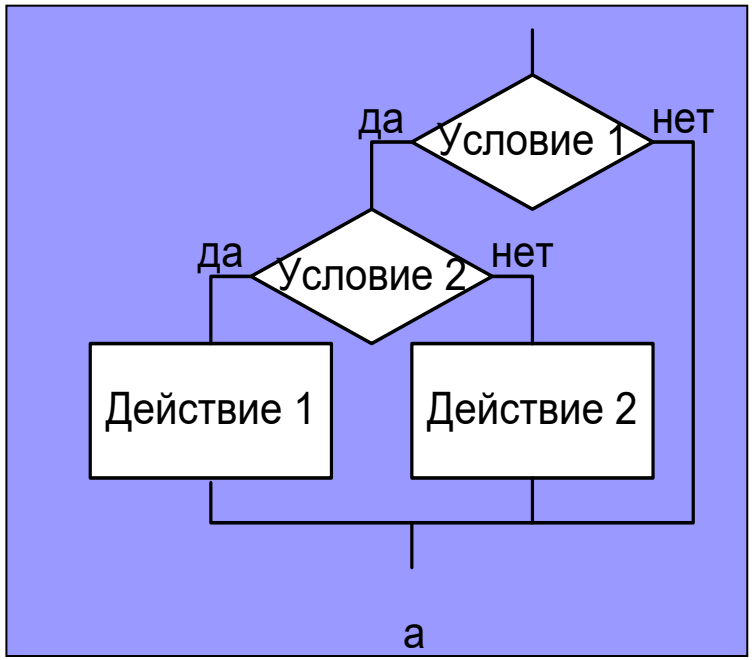



```
#include <iostream>
using namespace std;
int main()
{
    unsigned char ch;
    int n;
    cout << "Enter n:";
    cin >> n;
    if (n>=0 && n<=15){
        if (n < 10)
            ch ='0'+n;
        else ch ='A'+n-10;
        cout << "n = " << ch;
    }
    else cout << "Error!";
    return 0;
}
```

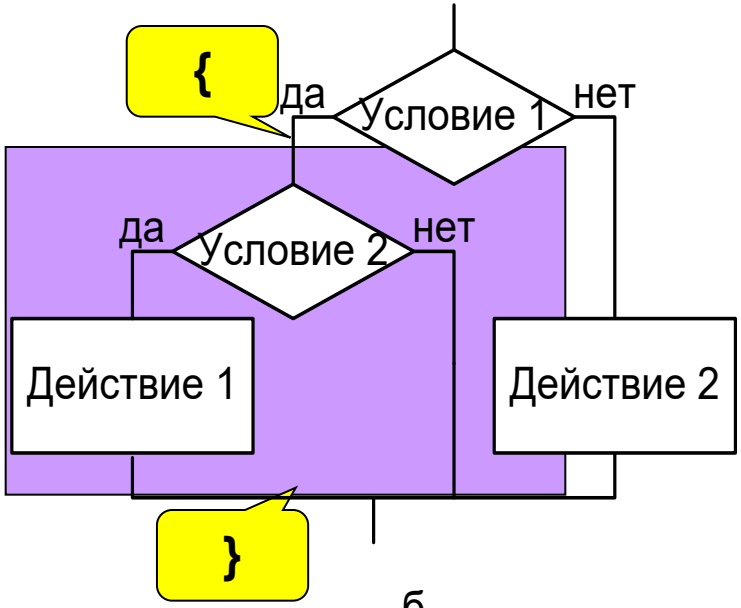
Правило вложения

```
if (Условие1)
        if (Условие2) (Действие1)
    else (Действие 2)
```

Предупреждение компилятора:
add explicit braces to avoid dangling else



а



б

По стандарту C++ ветвь else относится к ближайшему if.

Для реализации варианта *б* используют {…}:

```
if (Условие1)
    {if (Условие2) (Действие1) }
else (Действие 2)
```

2.3 Оператор выбора

Оператор позволяет программировать несколько вариантов решения.

```
switch (Выражение)
{
        case Элемент: {Оператор} { case Элемент: {Оператор} }
        [ default : {Оператор} ]
}
```

Пример:

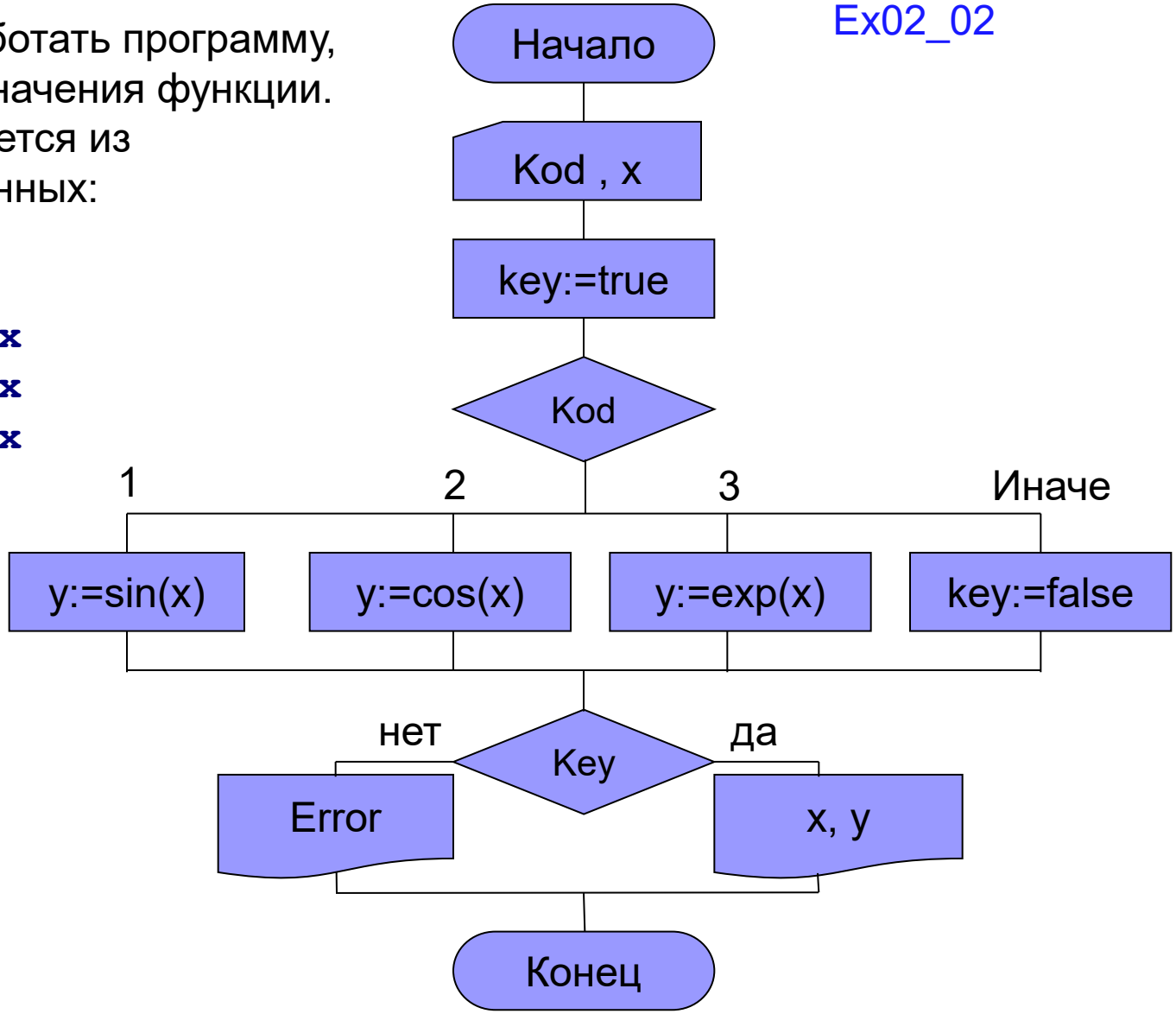
```
switch (n_day)
{ case 1:
  case 2:
  case 3:
  case 4:
  case 5: cout << "Go work!" << endl; break;
  case 6: cout << "Clean the yard and";
  case 7: cout << "relax!" << endl;
}
```

Программа вычисления значений функций

Ex02_02

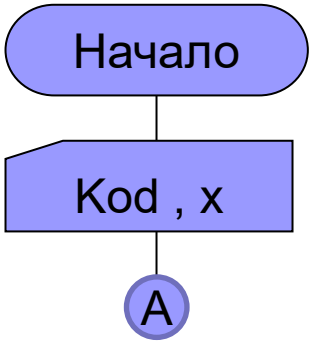
Задание. Разработать программу, вычисляющую значения функции. Функция выбирается из нескольких заданных:

```
Enter cod:
   1 – y=sin x
   2 – y=cos x
   3 – y=exp x
```

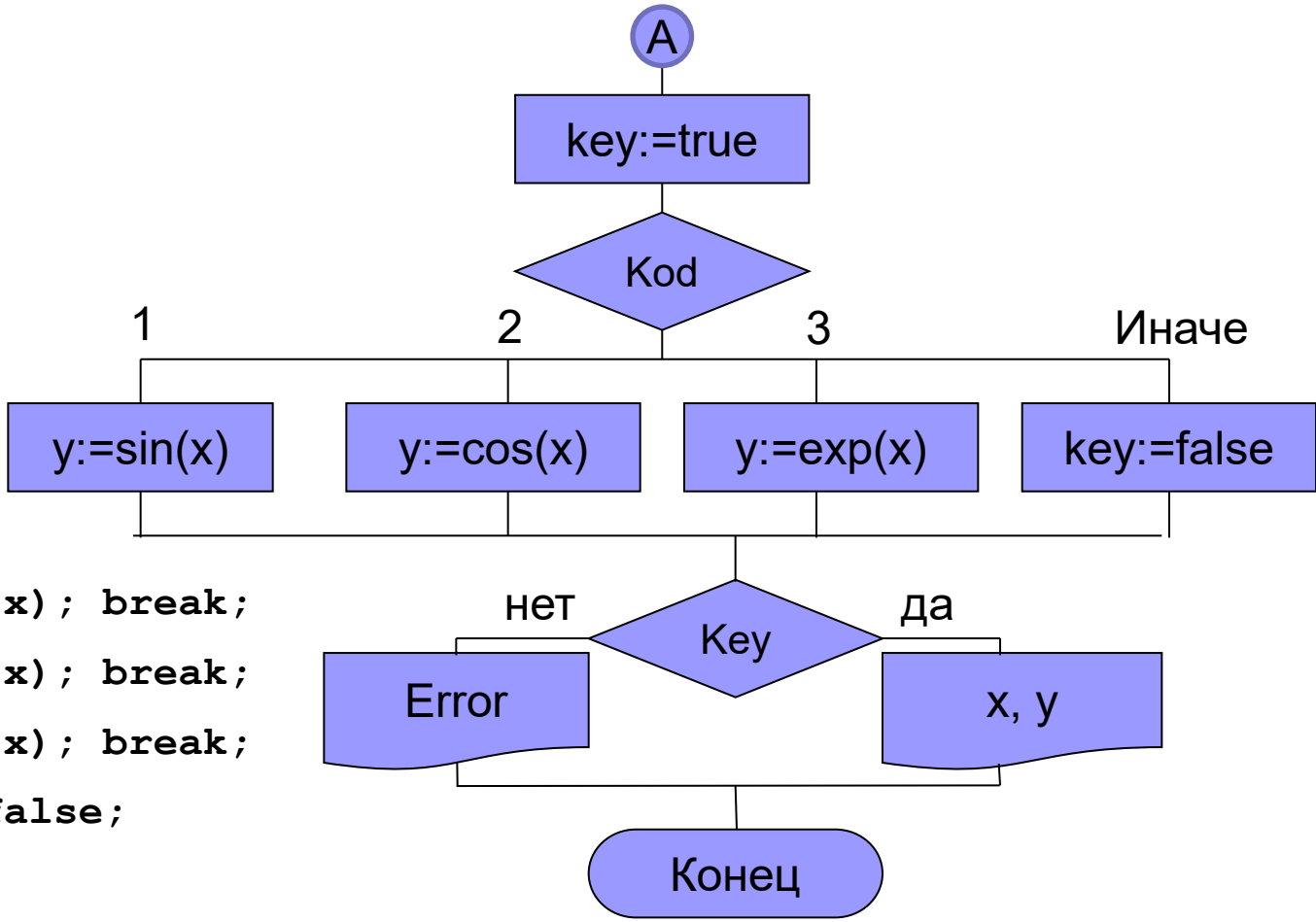


Программа вычисления значений функции

```
#include <stdio.h>
#include <cmath>
int main()
{
   int kod;
   bool key;
   float x, y;
   puts("Enter x:");
   scanf("%f",&x);
   puts("Enter kod:");
   puts("1 - y=sin(x)");
   puts("2 - y=cos(x)");
   puts("3 - y=exp(x)");
   scanf("%d",&kod);
```

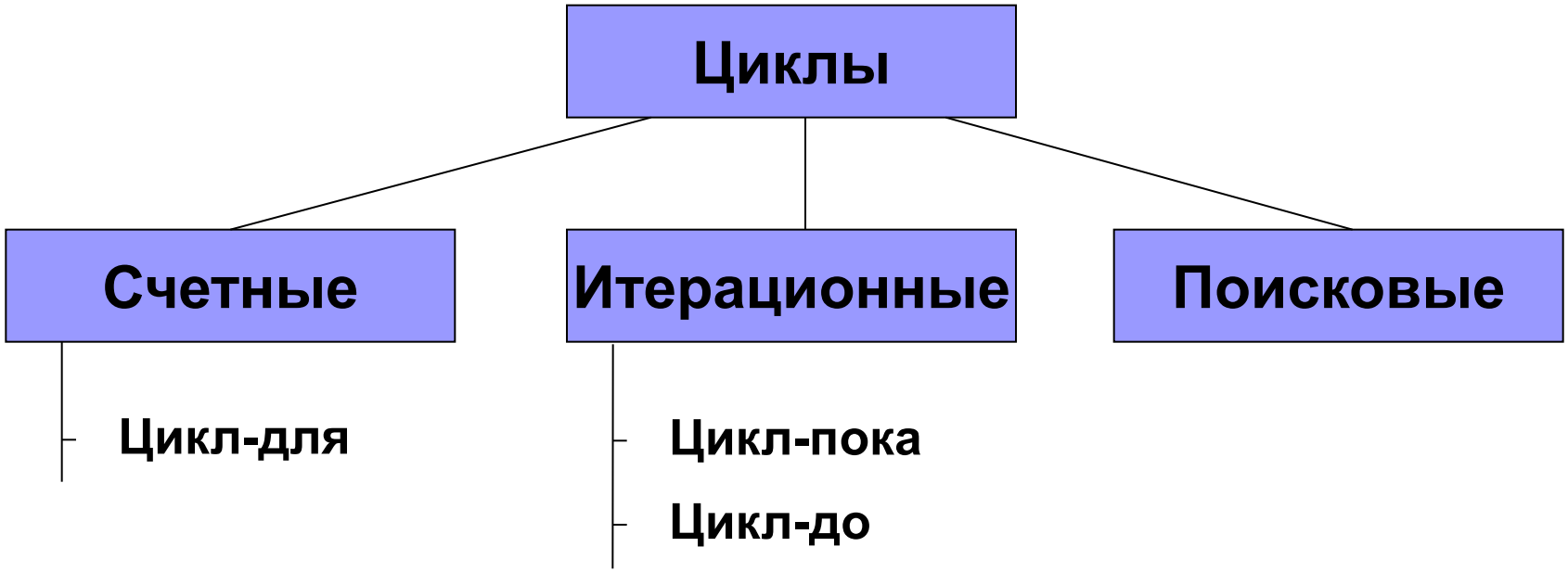


Программа вычисления значения функции (2)




```
key=true;
switch(kod)
{
     case 1: y=sin(x); break;
     case 2: y=cos(x); break;
     case 3: y=exp(x); break;
     default: key=false;
}
if (key) printf("x= %5.2f y=%8.6f\n",x,y);
    else puts("Error");
    return 0;
}
```

2.4 Операторы организации циклов

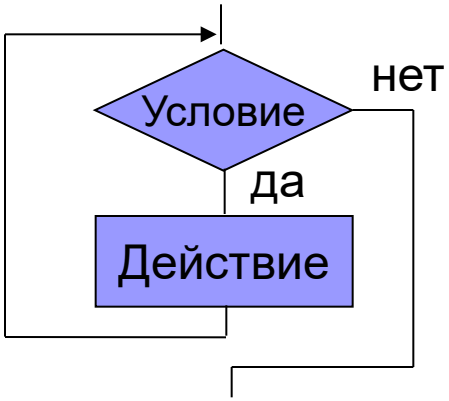


Счетный цикл – цикл, количество повторений которого известно или можно посчитать. Выход из такого цикла программируется по счетчику.

Итерационный цикл – цикл, количество повторений которого неизвестно или считается неизвестным при построении цикла. Выход из цикла осуществляется по выполнению или нарушению условия.

Поисковый цикл имеет два выхода – нашли и перебрали все, но не нашли.

Цикл-пока. Конструкция Цикл-while



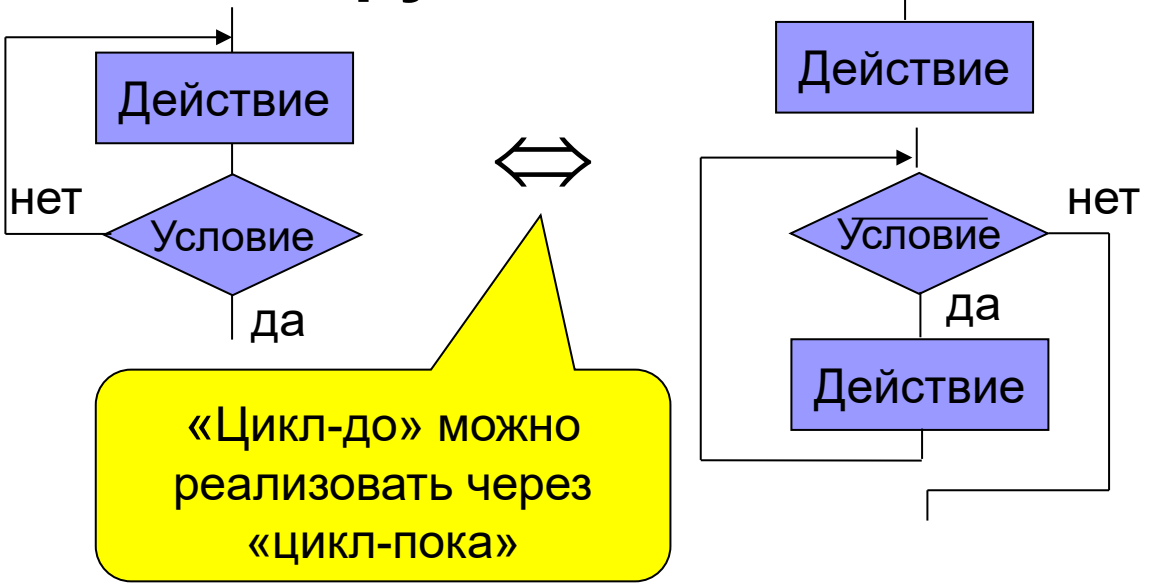
`while` (Выражение) Оператор

Цикл выполняется пока результат выражения - истина.

Пример:

```
while (fabs(e) >= 1e-5 )
{
    x = x + 1;
    e = e / 10;
};
```

Цикл-до. Конструкция Цикл-do-while



`do` Оператор `while` (Выражение) ;

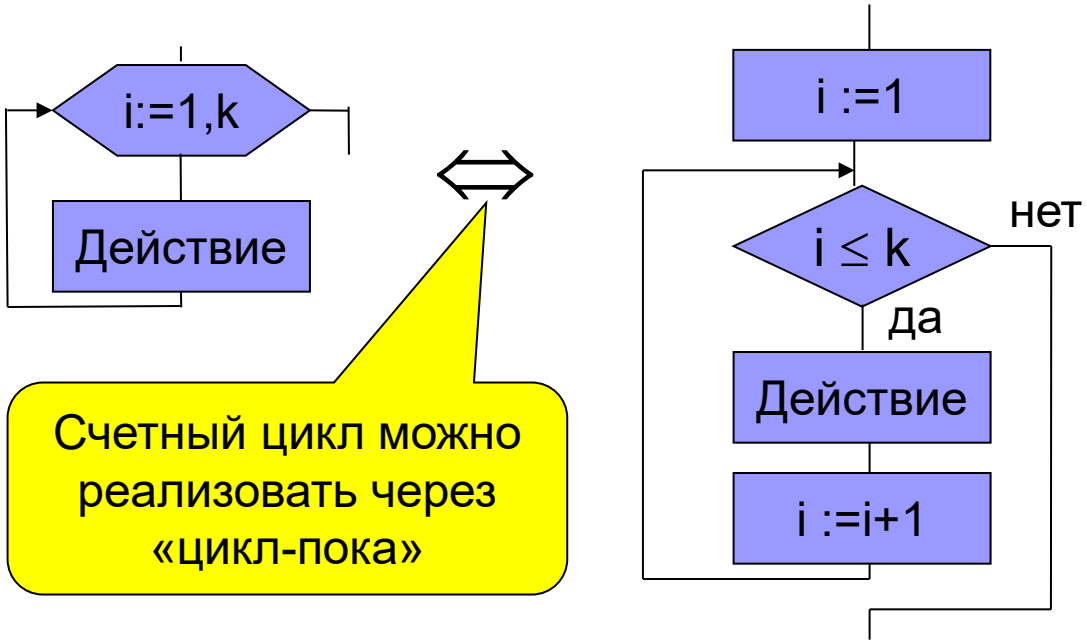
Цикл выполняется до тех пор, пока результат выражения не станет true.

Пример. Игнорировать ввод значения, выходящего за пределы заданного интервала:

```
do {
        printf("Enter value from %d to %d : ", low, high);
        scanf("%d ", &a);
    }
while (a < low || a > high);
```

Счетный цикл. Конструкция Цикл-for

В C++ счетный цикл реализуется посредством конструкции Цикл-for.



`for` ([Выражение1];[Выражение2];[Выражение3]) [Оператор]

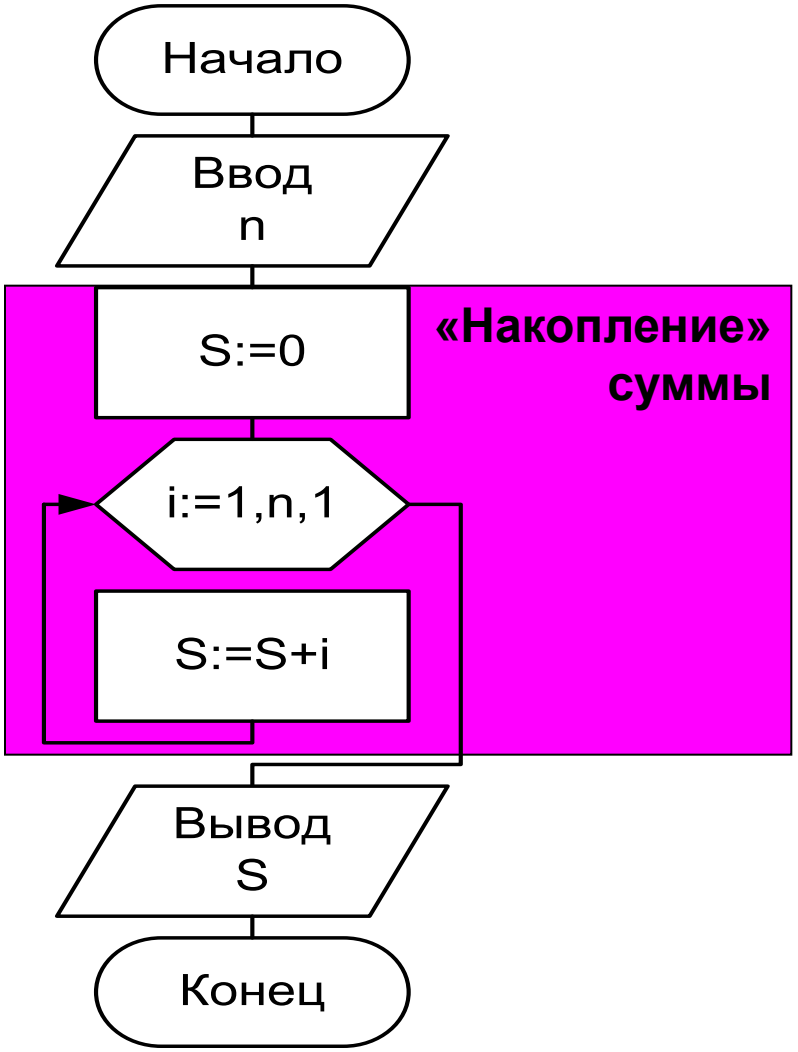
Эквивалентно:

```
Выражение1
while (Выражение2)
{
    Оператор
    Выражение3;
}
```

В C++ Цикл-for – частный случай цикла-пока, который обычно используется для реализации счетного цикла.

Суммирование натуральных чисел (Ex02_03)

Найти сумму первых N натуральных чисел.

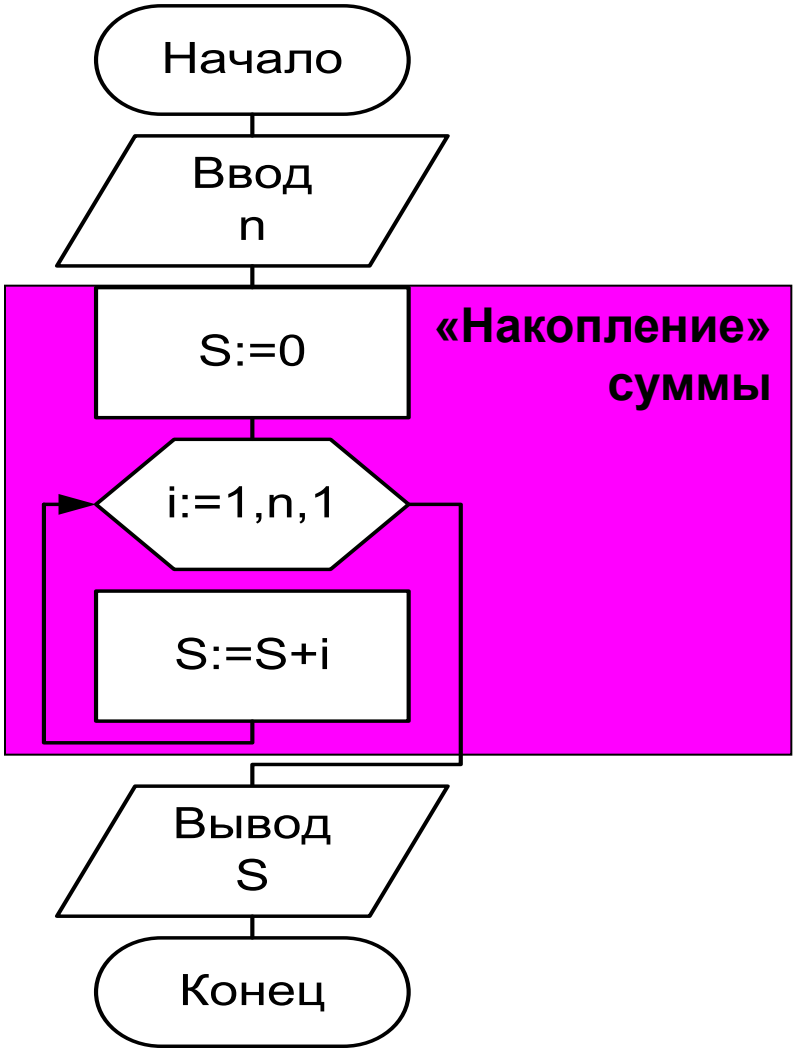



```
#include <iostream>
using namespace std;
int main()
{
    int n, s;
    cout << "Enter n: ";
    cin >> n;
    for (int i=1,s=0;i<=n;i++)
        s += i;
    cout << "Sum=" << s << endl;
    return 0;
}
```

Переменные i и s(!) локальны внутри цикла!!! Ошибка!!!

Суммирование натуральных чисел (Ex02_03)

Найти сумму первых N натуральных чисел.



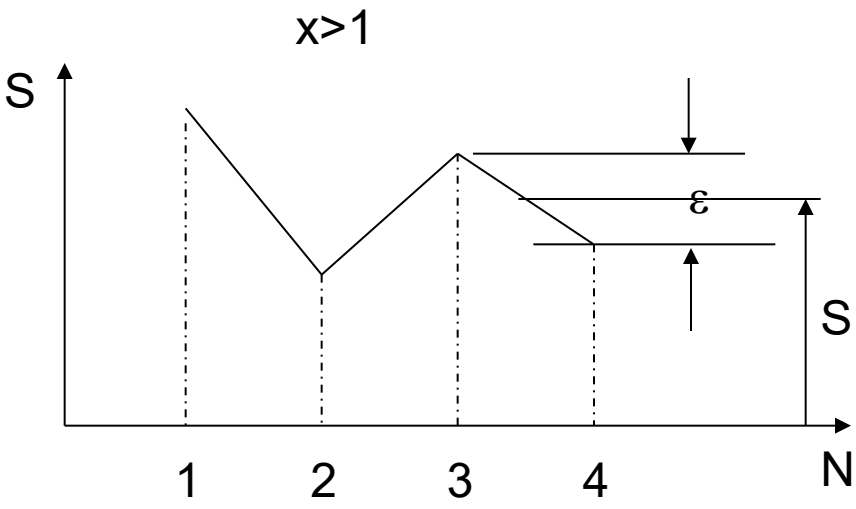

```
#include <iostream>
using namespace std;
int main()
{
    int n, s=0;
    cout << "Enter n: ";
    cin >> n;
    for (int i=1 ; i<=n ; i++)
        s += i;
    cout << "Sum=" << s << endl;
    return 0;
}
```

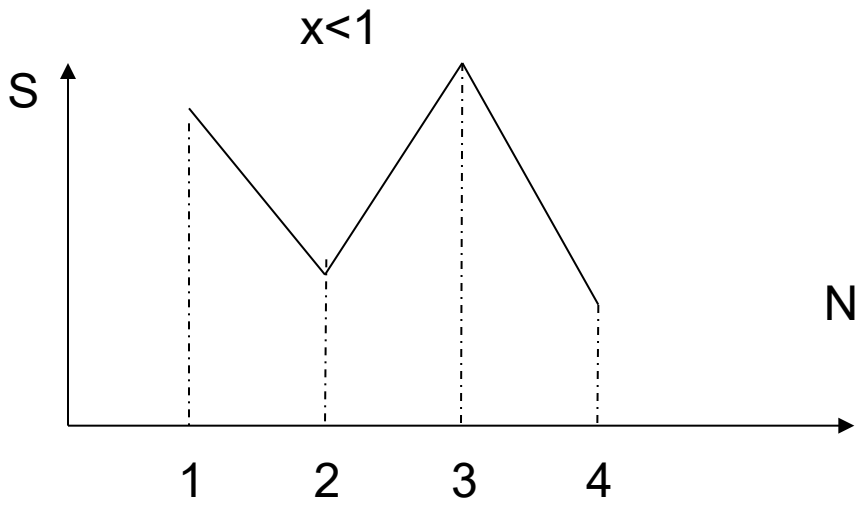
Переменная i локальна внутри цикла

Суммирование ряда (Ex02_04)

Определить сумму ряда

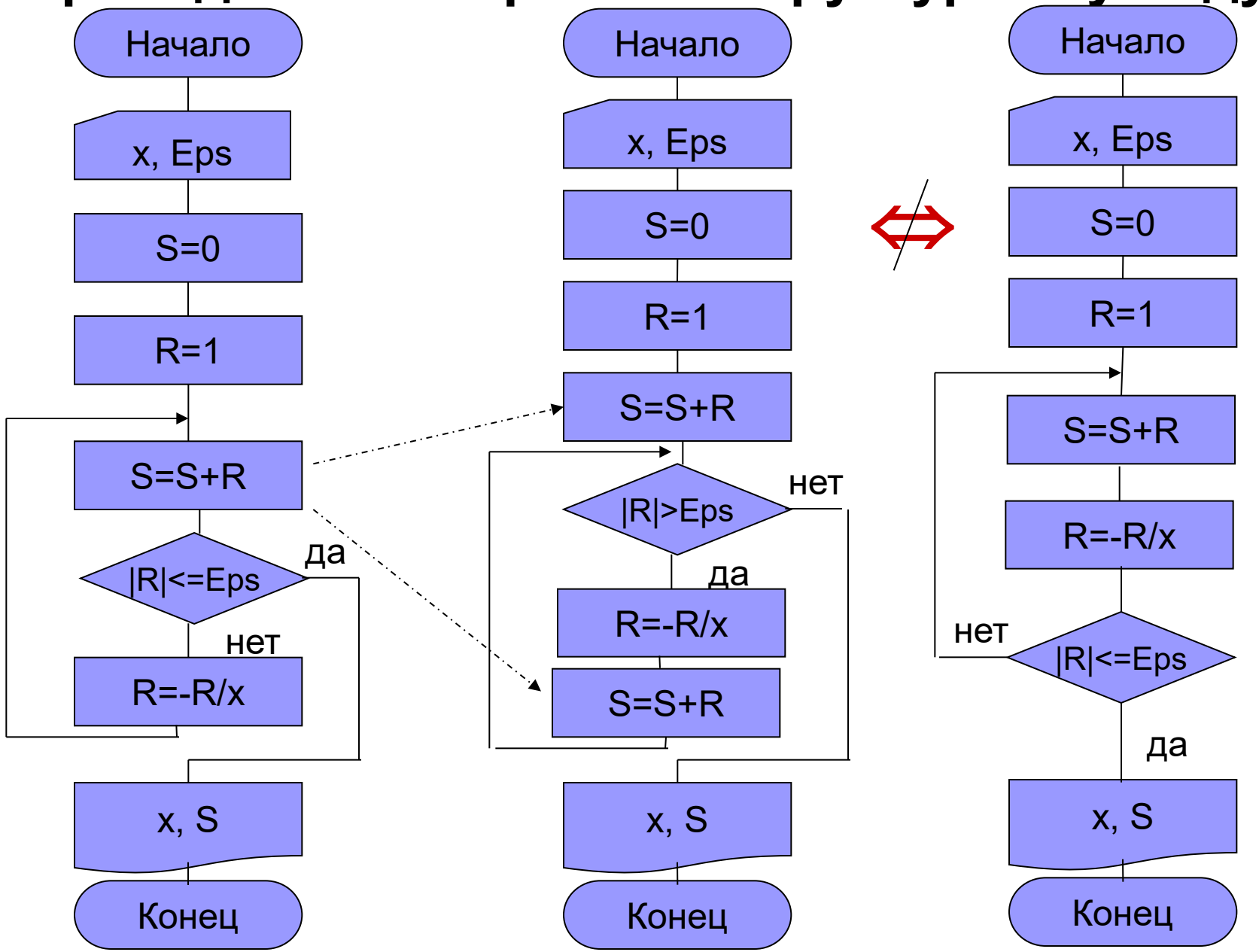
$S = 1 - 1/x + 1/x^2 - 1/x^3 + \ldots$ с заданной точностью ε.



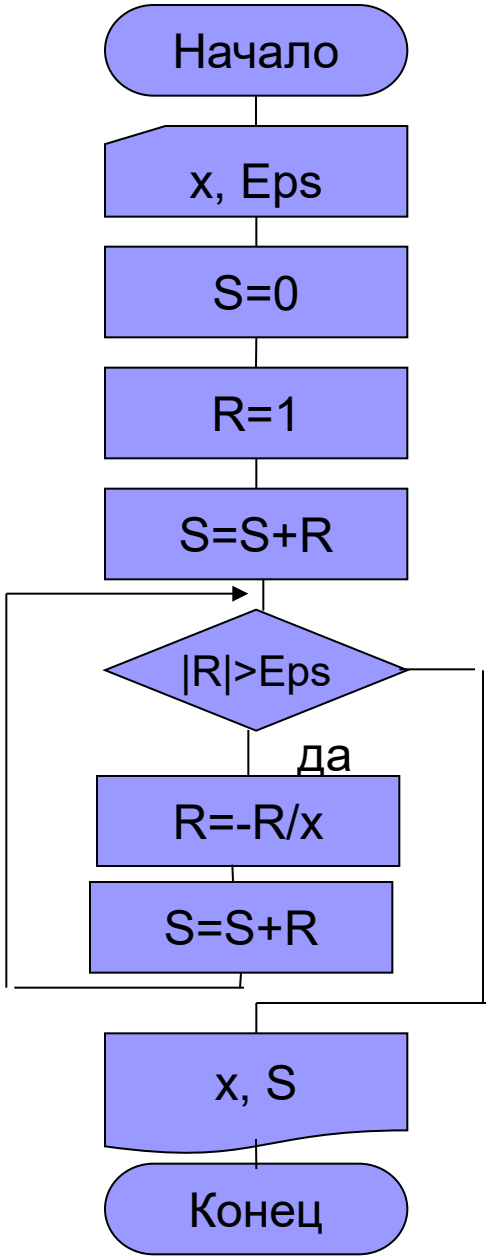


$$R_n = -R_{n-1}/x$$

Приведение алгоритма к структурному виду

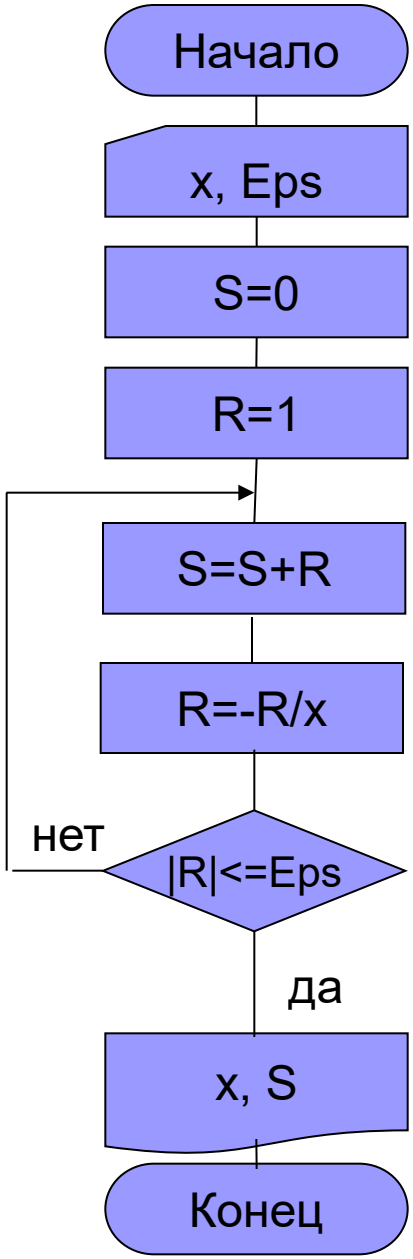


Вариант а




```
#include <stdio.h>
#include <cmath>
int main()
{
    float s, r,x,eps;
    puts("Input x, eps:");
    scanf("%f %f", &x, &eps);
    s=0;
    r=1; s+=r;
    while (fabs(r)>eps)
    {
        r=-r/x;
        s+=r;
    }
    printf("Result=%10.7f  r=%10.8f\n",
                                  s,r);
    return 0;
}
```

Вариант б

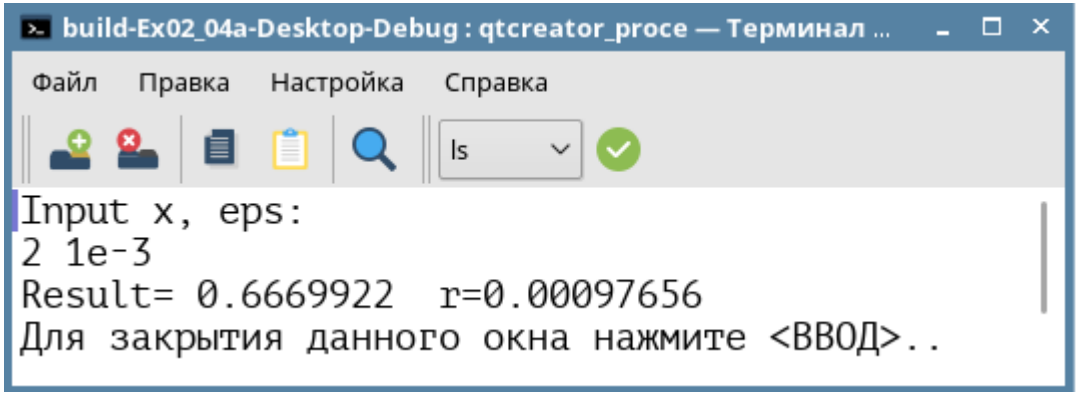


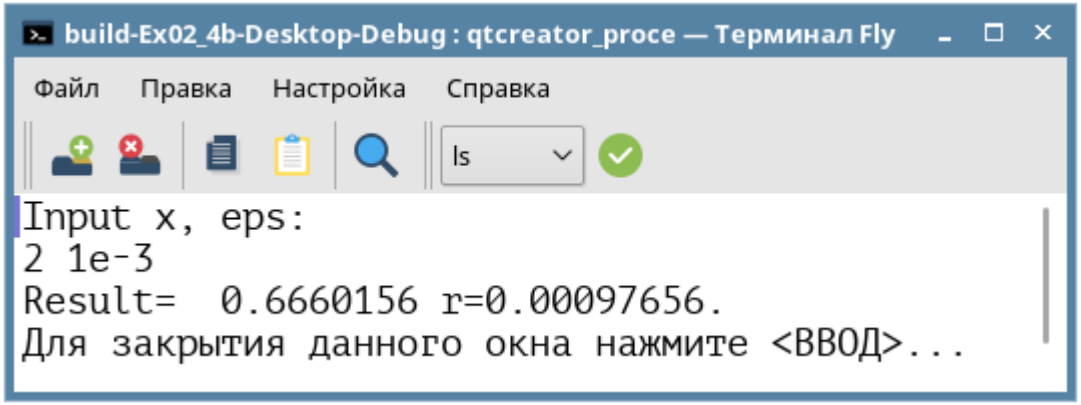

```
#include <stdio.h>
#include <math.h>
int main()
{
    float s, r,x,eps;
    puts("Input x, eps:");
    scanf("%f  %f", &x, &eps);
    s=0;     r=1;
    do
    {
        s+=r;
        r=-r/x;
    }
    while (fabs(r)>eps);
    printf("Result= %10.7f r=%10.8f.\n",
                                    s,r);
    return 0;
}
```

Сравнение результатов

Очередной элемент и там, и там – равны, но суммы – различны!!!

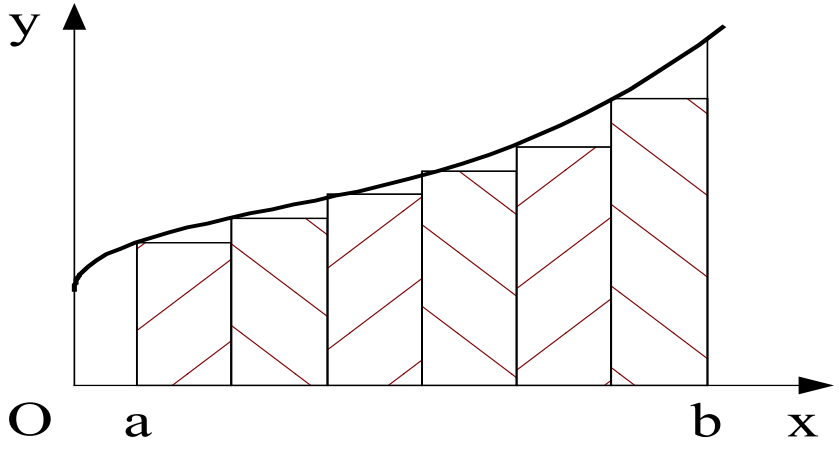
Так как в первом случае проверка выполняется после добавления элемента, а во втором – до.





2.5 Решение задач вычислительной математики

Задача Ex02_05. Вычислить определенный интеграл функции f(x) на интервале [a,b] методом прямоугольников с точностью δ.



n=6

Итак

$$S = f(x1)\times d + f(x2)\times d + f(x3)\times d + \ldots + f(xn)\times d = d\times\sum_{i=1}^{n} f(xi), \text{ где } d=(b-a)/n.$$

Увеличивая **n**, получаем приближения площади: $\mathbf{S_1, S_2, S_3 \ldots}$

Останавливаемся, когда $|S_k - S_{k+1}| < \delta$

Неформальное описание алгоритма

Алгоритм:

Шаг 1. Ввести a, b, δ.

Шаг 2. Задать число прямоугольников n:=10.

Шаг 3. Определить шаг d:=(b-a)/n.

Шаг 4. Определить площадь фигуры S1.

Шаг 5. Увеличить число прямоугольников вдвое n:=n*2.

Шаг 6. Уменьшить шаг вдвое d:=d/2.

Шаг 7. Определить площадь фигуры S2.

Шаг 8. Если Разность площадей меньше δ, то перейти к шагу 11

Шаг 9. Запомнить новое значение площади S1:=S2.

Шаг 10. Перейти к шагу 5.

Шаг 11. Вывести S1.

Конец.

Схема алгоритма (неструктурная и неэффективная)

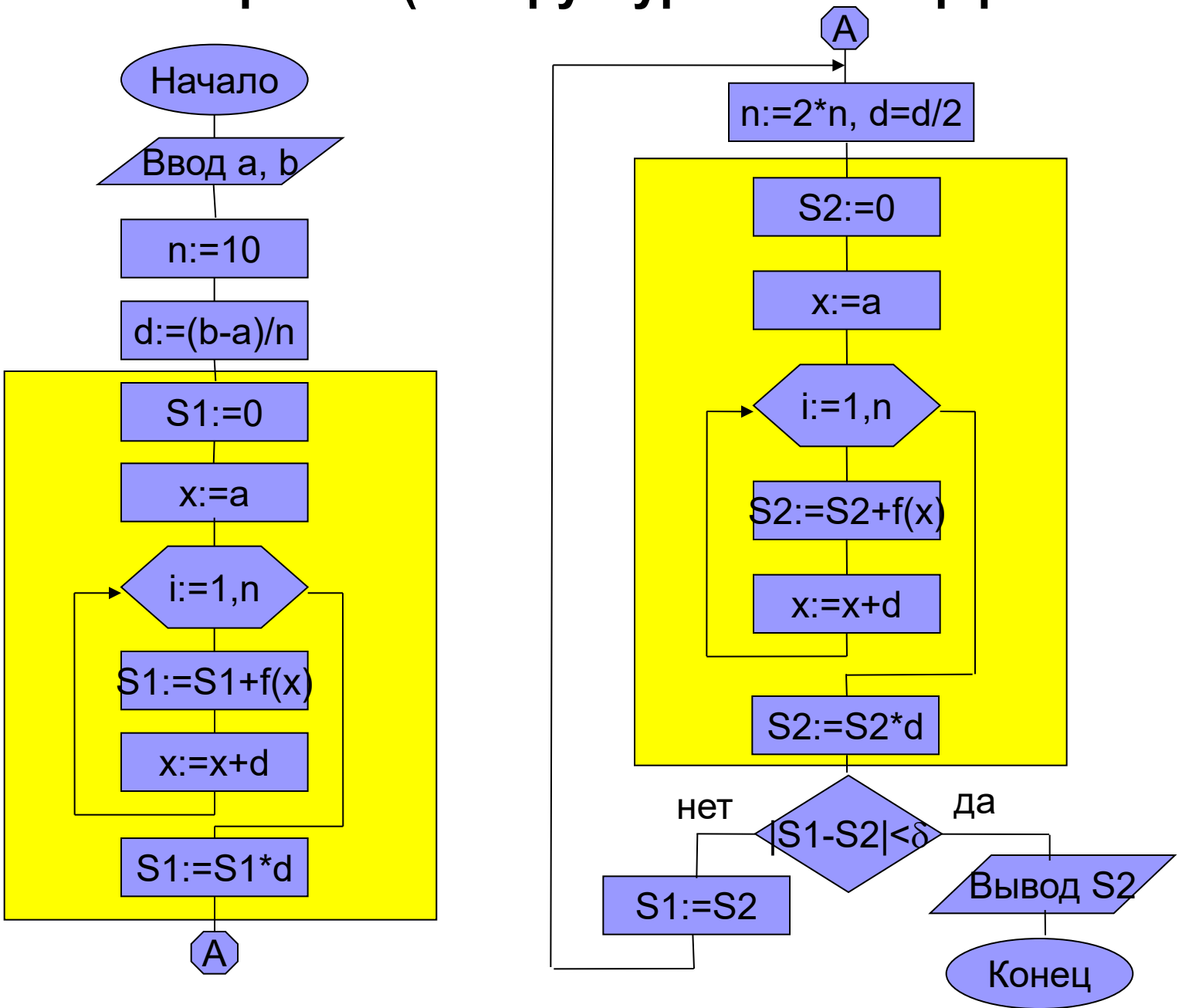
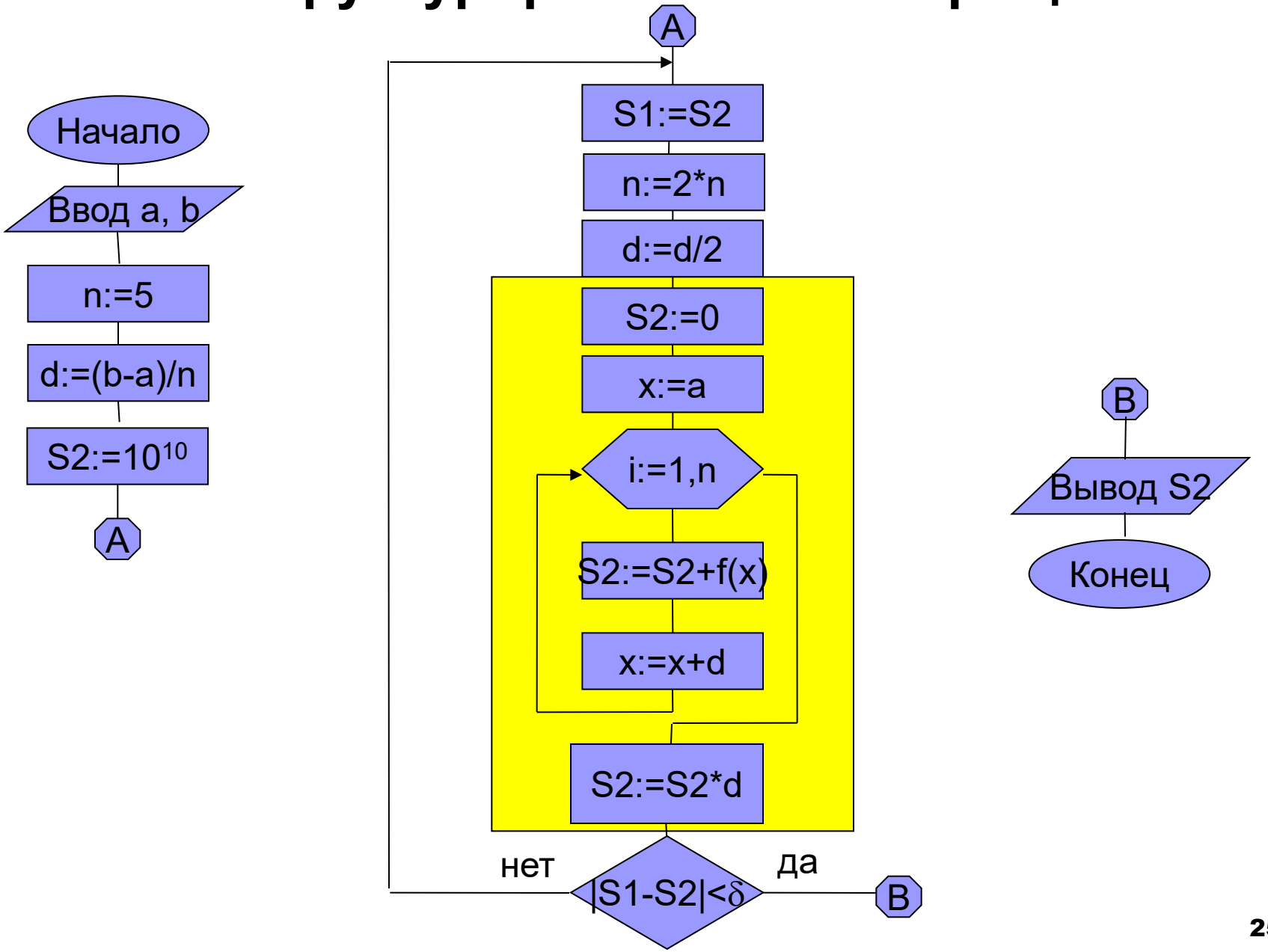


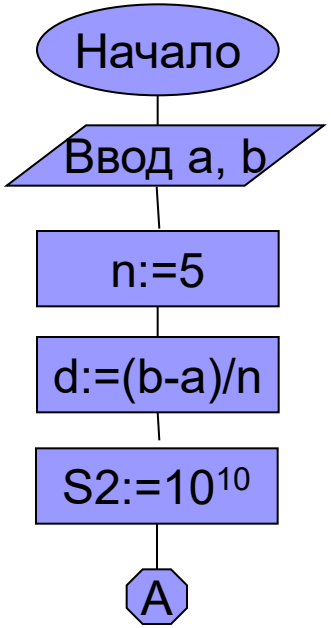
Схема структурированная и сокращенная



Программа

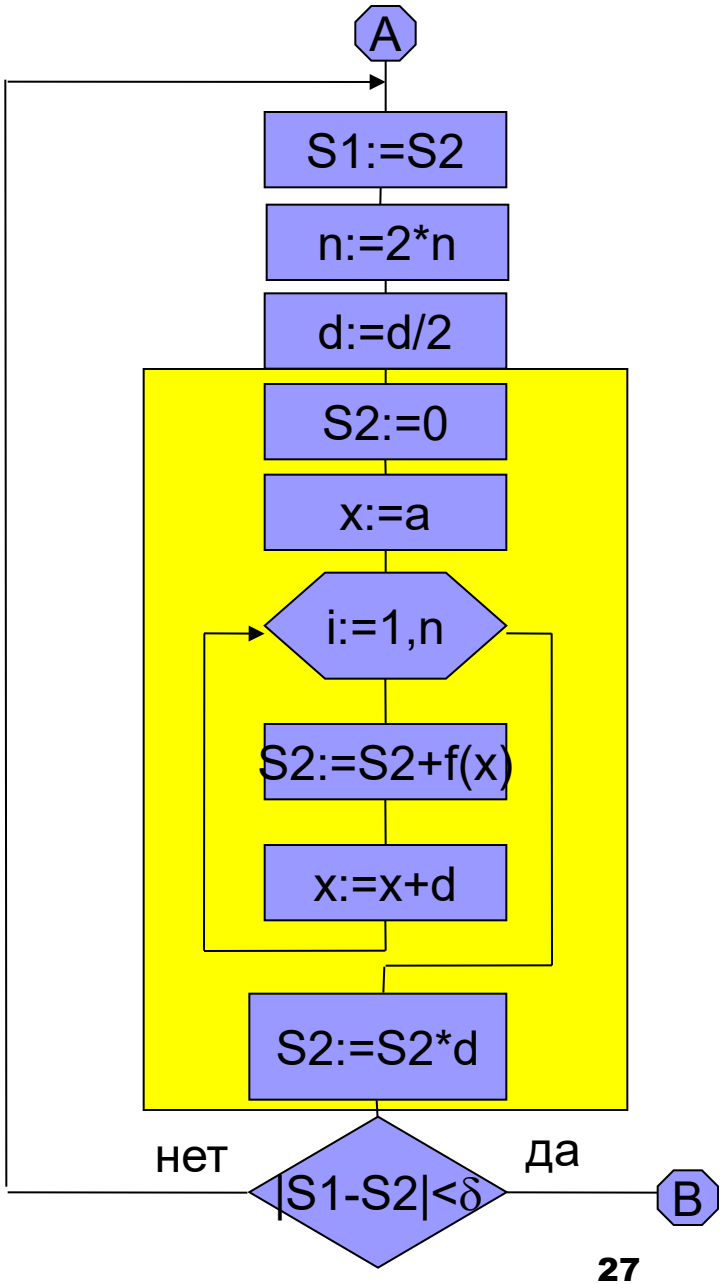
```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    int i,n;
    float s1,s2,x,a,b,eps,d;
    cout << "Enter a,b,eps:";
    cin >> a >> b >> eps;
    n=5;
    d=(b-a)/n;
    s2=1.0e+10;
```

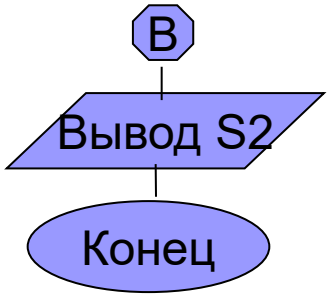


Программа (2)

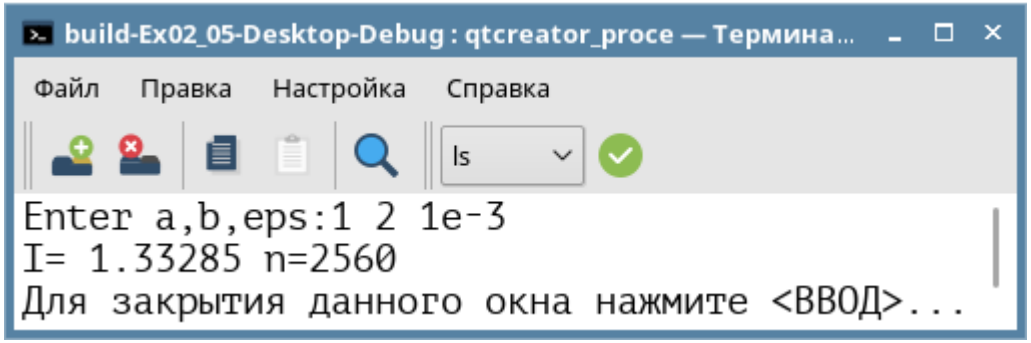
```
do
{
    s1=s2;   s2=0;
    n=n*2;   d=d/2;
    x=a;
    for(i=1;i<=n;i++)
    {
        s2=s2+x*x-1;
        x=x+d;
    }
    s2=s2*d;
}
while(fabs(s2-s1)>eps);
```



Программа (3)




```
   cout << "I= " << s2 << " n=" << n << endl;
   return 0;
}
```



2.6 Неструктурные операторы передачи управления

А. Оператор безусловного перехода goto

goto <Метка перехода>;

Пример:

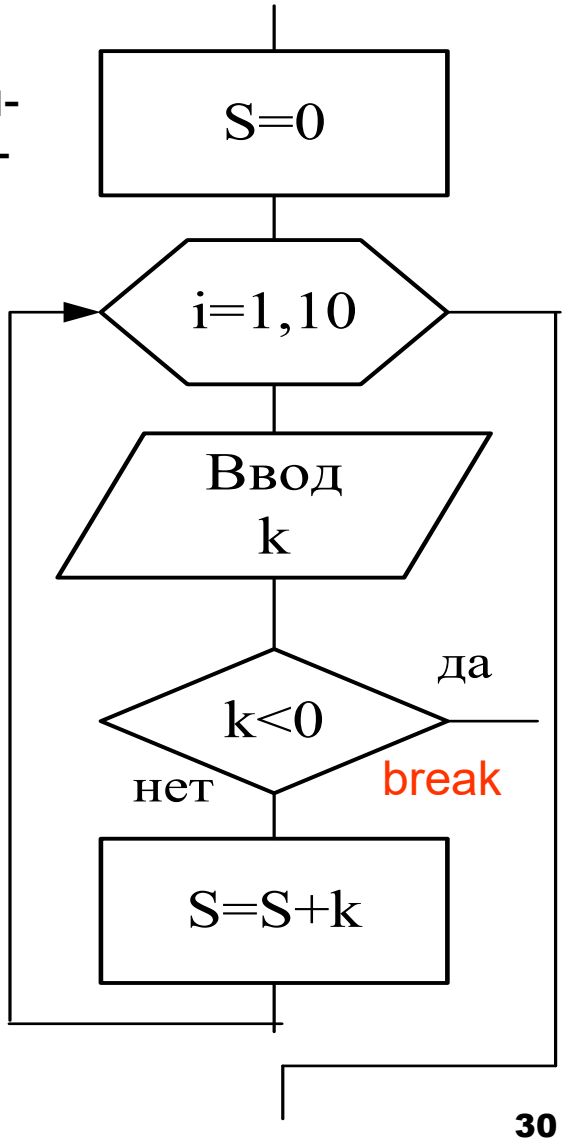
```
again: x=y+a;
            ...
       goto again;
```

Б. Оператор досрочного завершения break

break;

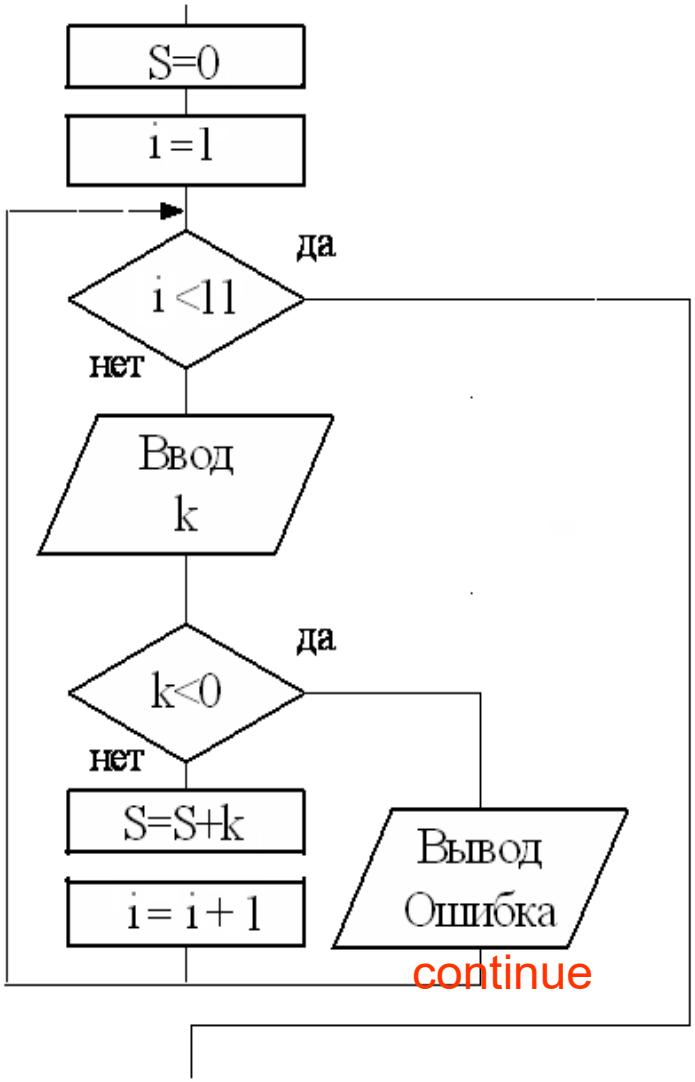
Пример Ex02_06. Суммирование до 10 чисел вводимой последовательности. При вводе отрицательного числа работа программы завершается.

```
#include <iostream>
using namespace std;
int main()
{   int s=0,k;
    cout<<"Input up to 10 numbers."<<endl;
    for (int i=1; i<11; i++)
    {
      cin >> k;
      if (k < 0) break;
      s+=k;
    }
    cout << "Result =" << s << endl;
    return 0;
}
```



В. Оператор продолжения continue

continue;

Пример Ex02_07. Программа суммирует 10 целых положительных чисел.

```
#include <iostream>
using namespace std;
int main()
{   int s=0,i=1,k;
    cout <<"Enter 10 numbers." << endl;
    while(i<11)
    {   cin >> k;
        if (k<0){   cout<<"Error."<<endl;
                    continue;
                }
        s+=k; i++;
    }
    cout << "Result =" << s << endl;
    return 0;
}
```